SAJI Technical Audit & Development Valuation Report

Prepared for: SAJI project stakeholders

Prepared by: Mobkazi - proposed engineering delivery partner

Date: 9 October 2026

Status: Client-facing assessment and proposed delivery approach

1. Executive assessment

SAJI contains a substantial existing software implementation that merits validation and completion rather than an assumed rebuild. The inspected source includes a multi-role web application, server-side business logic, a PostgreSQL schema, payment integration paths, geospatial discovery, messaging and operational modules.

Production readiness is NOT established. The present evidence supports a source-based architecture and feature inventory, not a completed comprehensive code audit, security clearance or end-to-end functional certification. No verified completion percentage or defensible monetary asset valuation is available yet.

Mobkazi proposes a paid validation and discovery phase, followed by a scoped stabilization engagement and controlled pilot. The agreed future direction remains separate frontend and backend repositories communicating through APIs. Baseline testing should precede extraction so existing behavior and defects can be distinguished from migration regressions.

Assessment area	Current conclusion
Source access and architecture inventory	Completed for the inspected scope
Comprehensive code and authorization audit	Pending; the reports do not establish exhaustive coverage
End-to-end functionality	Pending verification
Live hosting and integration connectivity	Unverified
Production readiness	Not established; readiness gates remain open
Development completion percentage	Undetermined
Replacement cost and existing asset value	Methodology defined; monetary conclusion pending evidence
Business market value	Outside the current technical evidence

2. Basis and limits of the assessment

This report consolidates the SAJI stack/architecture report and feature evidence matrix produced from D:\OLIVER\MAIN. The inspected Git commit is 3ad9644eedcb1749550ef55a87bb4565af6e636f, with 92 reachable commits. The working tree also includes four tracked POS-related edits and one untracked manual SQL file. Those changes require review and release reconciliation.

Source inspection established 138 Prisma model declarations and 156 files in lib/actions/. These are source counts, not a live database inventory or measures of delivered business value. Earlier reported PostgreSQL 17.6, 431 indexes and five deployed GiST indexes were not independently reverified.

The audit did not run full application acceptance tests, verify live payment settlement, inspect all authorization paths, measure hosting capacity or establish cloud-account ownership. Findings describe the inspected snapshot and require revalidation after later changes.

3. Confirmed stack and current architecture

Layer	Source-established technology
Web application	Next.js 16.0.10 App Router; React / React DOM 19.2.0
Language and styling	TypeScript; Tailwind CSS 4 range; Radix/shadcn-style UI; Ant Design 6.1.4
Server	Next.js server actions and API route handlers
Authentication	Auth.js / NextAuth 5.0.0-beta.31; Prisma adapter; JWT sessions
Data access	Prisma / Prisma Client 6.19.3
Database	PostgreSQL configured on Supabase; PostGIS query code
Discovery	Google Maps integration and geospatial SQL
Communication	Five-second message polling; WebRTC voice and polled signaling; STUN without identified TURN
Payments	M-Pesa Daraja, Stripe and PayPal integration paths
AI and notifications	Google GenAI, Anthropic, SMTP/Nodemailer, Infobip 2FA and Web Push
Observability	Sentry and Vercel Analytics configuration

SAJI currently runs as ONE full-stack Next.js application. Customer, provider, shopkeeper, distributor, agent and staff/admin portals are routes within that application. Business logic resides in lib/actions/ and core helper modules; HTTP endpoints reside in app/api/. Prisma connects server operations to the separately configured PostgreSQL service.

Current flow: Browser -> Next.js UI/server operations -> Prisma -> PostgreSQL/PostGIS. Server operations also connect to external providers. A dependency or configuration entry establishes an implementation reference, not successful live operation.

The canonical local remote points to https://github.com/OliverYoung-dev/SAJI-3.git. Active production hosting remains unconfirmed: standalone configuration references self-hosting/cPanel/Passenger/HostPinnacle, while Vercel deployment and scheduled-job configuration also exists.

4. Functional status and engineering findings

Area	Evidence and outstanding work
Authentication and role access	Authentication implementation exists; test recovery, OAuth, sessions, role isolation and every sensitive server operation
Discovery and location	PostGIS queries exist; verify deployed extension/indexes, address accuracy and radius behavior
Messaging	Polling implementation exists; verify two-user delivery, participant isolation and reconnect behavior
Voice calls	WebRTC/signaling exists; verify cross-network audio and establish whether TURN relay is required
Payments	Integration paths exist; missing M-Pesa credentials can simulate completion. Validate sandbox callbacks, idempotency and reconciliation before live use
Airtel	Sample configuration describes the gateway as unimplemented; unavailable checkout paths need review
Wallet and escrow	Modules exist; ledger integrity, concurrent settlement, refunds and dispute transitions remain unverified
POS/inventory	Database-backed operations exist with local unreleased edits; verify stock consistency, returns, offline sync and schema alignment
KYC and administration	Modules exist; verify document privacy, approval authority and coverage of transaction gates
AI/email/SMS/push	Provider modules/configuration exist; real connectivity, delivery and failure behavior remain unverified

Area	Evidence and outstanding work
Reporting	Reconcile totals and exports against underlying transactions and permissions
Deployment and cron	Establish actual host, packaging, runtime compatibility, scheduler, secrets and rollback procedure

Material source findings: TypeScript build errors are ignored in next.config.mjs; the referenced standalone postbuild script is empty; no .github directory or conventional test suite was identified by filename searches; manual SQL records do not form an automatically replayed migration history; and the local environment template is not tracked. The README also describes an earlier frontend-only state and needs updating.

These findings are delivery risks and verification gaps. They do not establish that every affected workflow fails, nor do they establish exploitable vulnerabilities without further testing.

5. Production readiness decision

Recommendation: do not represent SAJI as production-certified or release unrestricted financial workflows on the strength of these reports. A controlled staging validation should establish the following acceptance gates.

Gate	Evidence required for acceptance	Lead responsibility
Reproducible release	Clean checkout builds, documented runtime, type checks, dependency setup and deployment artifacts	Mobkazi engineering
Core workflows	Agreed customer/provider/staff journeys pass with real persistence; no undisclosed mock or simulated production behavior	Mobkazi QA + SAJI product owner
Access and privacy	Server-side authorization and tenant/user isolation checks; secrets handling and KYC-file access validated	Mobkazi engineering/security review
Money integrity	Sandbox transaction lifecycle, duplicate callbacks, ledger reconciliation, refunds and settlement tested	Mobkazi + SAJI finance + payment provider
Data recovery	Reviewed migrations, verified backup restoration and documented recovery objectives	Mobkazi + hosting/database owner
Operations	Health checks, alerts, scheduled jobs, logs, incident ownership and rollback exercised	Mobkazi operations + SAJI
Capacity	Load test against an agreed concurrent-user and transaction profile; bottlenecks addressed	Mobkazi engineering
Ownership	Business-controlled repository, hosting, database, domain and provider access verified	SAJI owner with Mobkazi support
Release approval	No unresolved release-blocking defects in agreed launch scope; known limitations recorded and accepted	SAJI product owner

Legal, commercial and payment-provider requirements should be confirmed by SAJI and its relevant advisers/providers where applicable. This technical report does not certify regulated financial operations.

6. Mobkazi delivery approach

Phase A: validation and discovery

Establish authorized read-only account access, preserve current changes, reproduce the application in staging and identify business-critical journeys. Test authentication, role isolation, payments in sandbox, wallet/escrow and POS. Inspect live schema and deployment configuration safely. Record each feature as verified, implemented awaiting testing, partial, demonstration/mock, absent or blocked.

Deliverables: accepted feature matrix with test evidence; deployment and ownership map; prioritized defect backlog; API/consumer inventory; release scope; and a priced implementation proposal. This phase replaces assumptions with evidence

before a fixed completion quotation.

Phase B: stabilization and pilot readiness

Address release-blocking defects in the agreed scope, establish meaningful automated checks, restore type-check visibility, document deployment packaging and baseline migrations, verify payment safeguards and configure monitoring/recovery. Run acceptance tests and a limited pilot with agreed users and explicit operating limits.

Deliverables: test results, reproducible deployment, operational runbook, known-issue register and pilot release recommendation. Security sign-off must name its scope; passing functional tests is not equivalent to an exhaustive penetration test.

Phase C: independent frontend/backend repositories

Create SAJI-owned saji-frontend and saji-backend repositories and deployments. Keep Next.js for the frontend; choose the backend framework during technical design rather than assuming one from this report. Define versioned API contracts and an authentication model before moving database-dependent server actions.

Move persistence, business validation, authorization, migrations, payment callbacks and private integrations to the backend. Replace frontend server-action imports with API clients. Migrate in bounded slices, compare behavior with baseline tests and preserve rollback. Financial workflows require dedicated regression and reconciliation checks.

Deliverables: API specification, independent pipelines, domain endpoints, frontend API client, migration documentation and acceptance evidence. A repository split alone does not demonstrate readiness.

Phase D: launch and support

Complete capacity testing, restore/rollback exercises and acceptance review. Execute a controlled production cutover with monitoring and named incident contacts. Agree support hours, response commitments, maintenance scope and recurring costs separately.

7. Provisional effort and schedule

These are Mobkazi planning allowances for discussion, not measured task estimates, a fixed-price quotation or a commitment. One person-day assumes eight working hours. They assume a tightly agreed MVP scope, access to accounts, timely decisions and no major redesign or data remediation.

Work package	Provisional effort	Intended output
A: validation and discovery	8-15 person-days	Evidence-backed backlog and implementation quotation
B: stabilization of agreed pilot scope	20-45 person-days	Tested pilot candidate and operations baseline
C: frontend/backend API extraction	25-50 person-days	Independent repos/deployments and verified API boundaries
D: launch validation and handover	5-10 person-days	Launch review, cutover and documentation
Arithmetic total, if all phases selected	58-120 person-days	Subject to re-estimation after Phase A

Phase A can provisionally occupy 1-2 calendar weeks with two available delivery contributors and prompt access. For the whole selected program, 8-16 calendar weeks is a planning allowance for two engineers with part-time QA/operations; dependencies, provider approvals and client decisions can extend it. Effort is not interchangeable with calendar duration, and staff availability is not established here.

The stabilization and extraction phases share some setup and tests. Their boundaries must be reconciled in the final backlog to

avoid double charging. The estimate excludes native mobile apps, extensive UI redesign, new business modules, large-scale data cleanup, formal independent penetration testing and provider onboarding fees unless specifically added.

8. Development valuation and quotation method

Replacement cost

Estimate what it would cost to recreate the VERIFIED functionality at an agreed quality level: accepted feature inventory -> design/build/test effort -> Mobkazi agreed rates -> integrations/operations/documentation costs. Existing source counts and commit history are not substitutes for this estimate. The completion allowances in section 7 are not a replacement-cost valuation.

Existing software asset value

Assess usable code, architecture, documentation, ownership/IP rights, maintainability and verified functionality. Adjust for unfinished work, security exposure, migration burden and dependencies. Cash-flow or market evidence may affect value, but no monetary asset conclusion can be justified from this source snapshot alone. The asset value is not automatically replacement cost minus the remaining development quotation.

Business market value

Requires business information: active users, revenue, retention, acquisition costs, margins, contracts, liabilities and market assumptions. Mobkazi can supply technical evidence; a formal business valuation requires an appropriately scoped separate engagement.

How Mobkazi should price this engagement

Offer Phase A first as a clearly scoped paid engagement. After acceptance, price the stabilized launch scope and optional API extraction separately, using the verified backlog. Use milestone acceptance and written change control for new features. If uncertainty remains high, use capped time-and-materials rather than describing a provisional allowance as a fixed total.

Let R be the agreed blended day rate in KES. Preliminary labor budgeting is:

Selected scope	Labor-budget formula
Phase A only	8R-15R
Stabilized monolith pilot: A+B+D	33R-70R
Full program including split: A+B+C+D	58R-120R

No Mobkazi rate, tax treatment or commercial markup has been supplied, so this report intentionally gives formulas rather than an invented KES price. Add approved third-party costs and a disclosed contingency allowance to the commercial proposal; any contingency percentage must be agreed after discovery. Separate VAT/taxes, payment terms and support fees from engineering labor.

The client quotation should state deliverables, acceptance criteria, assumptions, exclusions, dependency owners, stage payments, change control and the duration of any defect-remediation period. Business approval is required for the final scope and contract terms.

9. Proposed infrastructure architecture

Use separate development, staging and production environments. For the current pilot, deploy the existing Next.js application with a supported Node runtime, a separate PostgreSQL service and configured provider sandbox accounts. Select one documented deployment method and verify packaging rather than relying on conflicting hosting references.

For the future split, route browser traffic to the frontend, and frontend API requests to the backend. Only the backend connects to PostgreSQL and holds private integration credentials. Place schema changes and scheduled workers under backend operational ownership. Define cookie/CORS/CSRF behavior if origins differ and use versioned API schemas or generated clients to coordinate independent releases.

Infrastructure component	Proposed responsibility and sizing approach
Frontend hosting	Next.js-compatible hosting/CDN; measure page load, caching and server-render demand
Backend hosting	Node-compatible API runtime; size from measured requests, latency and database concurrency
PostgreSQL/PostGIS	Separate managed database; verify connections, pooler behavior, region, backups and capacity
User-file storage	Private object storage for sensitive documents, controlled URLs and retention; audit existing persistence first
Job scheduler	Explicit authenticated cron/worker scheduling for digests and expiry tasks
Voice relay	Assess TURN service after cross-network tests; meter bandwidth and reliability
Monitoring	Application errors, uptime, logs, alerts and payment reconciliation signals
Secrets	Environment-specific secret management; no private keys in frontend bundles or Git
Delivery pipeline	Independent checks/deployments, reviewed schema changes and rollback procedure

Actual CPU/RAM, hosting provider, region and scaling limits must follow measured workload and confirmed platform capabilities. No Dockerfile or compose configuration was identified in the earlier source inventory; container availability is not established. Containers may be introduced for reproducibility if the selected host supports them.

Recurring budget must itemize frontend/backend hosting, database/backup/storage, bandwidth, TURN, monitoring, Maps, AI, SMS/email, domain and provider transaction charges. Obtain dated supplier quotations after selecting providers and projected usage. This report does not assert current supplier prices or a monthly operating total.

10. Business ownership and engineering handover

SAJI should own the GitHub organization and production assets, with individual accounts and documented team permissions. Preserve the existing repository history during transfer; maintain it as a transitional saji-platform repository while extracting saji-frontend and saji-backend. Confirm destination names and authorized owners before action.

GitHub ownership transfer does not transfer Supabase, hosting, domains, OAuth or payment-provider accounts. Inventory each asset, billing owner, access permissions, renewal responsibility and recovery contact. Mobkazi should receive scoped delivery access while SAJI retains business ownership and recovery control.

11. Proposed immediate decision

Approve a defined Phase A scope and Mobkazi commercial rate, nominate a SAJI product/acceptance owner and provide authorized staging/account access. Mobkazi then returns an evidence-backed feature status, prioritized launch backlog and priced milestone proposal. Complete baseline verification before committing to the full API extraction budget.

This approach recognizes SAJI's existing engineering investment, makes remaining work reviewable and preserves the agreed future architecture without claiming unverified functionality or valuation.

12. Supporting documents

- SAJI-STACK-ARCHITECTURE-AND-GITHUB.pdf: stack, source locations, architecture and ownership/extraction plan.

- SAJI-FEATURE-EVIDENCE-MATRIX.pdf: provisional feature evidence and required verification.

This report is a technical assessment and planning proposal. It is not a production certification, formal security clearance, binding development quotation or independent business valuation.